

# NeuroSky MindWave Mobile Android SDK

## Developer Guide · v2.0.1

---

### Table of Contents

1. [Overview](#)
  2. [How It Works — Architecture](#)
  3. [Requirements](#)
  4. [Installation](#)
  5. [Permissions](#)
  6. [Quick Start](#)
  7. [EEG Data Model](#)
  8. [EEG Frequency Bands Explained](#)
  9. [Signal Quality](#)
  10. [Commands](#)
  11. [Simulator — Develop Without Hardware](#)
  12. [Error Handling & Reconnection](#)
  13. [Advanced Patterns](#)
  14. [Troubleshooting](#)
  15. [Testing](#)
  16. [API Reference](#)
- 

## 1. Overview

The **NeuroSky MindWave Mobile Android SDK** is a modern Kotlin library that lets you read real-time EEG (electroencephalography) data from a NeuroSky MindWave Mobile headset on Android 6.0 (API 23) or later — with zero dependency on NeuroSky's legacy ThinkGear Connector (TGC) software.

### Why this SDK exists

The official NeuroSky Android SDK requires TGC running on a paired PC, communicating over TCP, and is built on a legacy Java API that does not work well with modern Android development patterns (Jetpack, Coroutines, Flow, ViewModel).

This SDK eliminates TGC entirely by communicating directly with the MindWave Mobile hardware via Android's Bluetooth stack. It is built on Kotlin Coroutines and Flow — the standard async primitives in modern Android.

### Key features

| Feature                  | Description                                                                          |
|--------------------------|--------------------------------------------------------------------------------------|
| No TGC dependency        | Communicates with hardware directly via Android Bluetooth APIs                       |
| BLE + BT Classic         | BLE by default; BT Classic available for noisy RF environments                       |
| Kotlin Coroutines & Flow | <code>Flow&lt;BrainWaveData&gt;</code> — integrates naturally with Jetpack lifecycle |
| Built-in Simulator       | Full data simulation without any hardware                                            |
| JitPack distribution     | One-line Gradle dependency, no local setup                                           |

Android 6.0+ (API 23+)

Wide device coverage

## What you can measure

The MindWave Mobile headset contains a single dry electrode on the forehead (FP1 position) and a reference clip on the ear. From this single channel, the ThinkGear ASIC chip on board computes:

- **Raw EEG waveform** — 512 samples/sec, signed 16-bit values
- **8 frequency band powers** — Delta, Theta, Alpha (Low/High), Beta (Low/High), Gamma (Low/Mid)
- **eSense™ Attention** — NeuroSky's proprietary attention index (0~100)
- **eSense™ Meditation** — NeuroSky's proprietary relaxation index (0~100)
- **Eye blink detection** — intensity 0~255
- **Signal quality** — 0 (perfect contact) to 200 (no signal)

## 2. How It Works — Architecture



```
| Your App |  
| .collect { } |  
└──────────┘
```

## BLE vs BT Classic — internal differences

**BLE (Bluetooth Low Energy) path:** The MindWave Mobile exposes three BLE GATT characteristics. The SDK subscribes to notifications on the eSense and RawEEG characteristics, then writes the handshake command byte to start data flow. No Android pairing is required.

**BT Classic (RFCOMM SPP) path:** The MindWave Mobile emulates a serial port (SPP UUID 00001101-... ). The SDK opens a `BluetoothSocket` and reads a continuous byte stream. `ThinkGearParser` synchronizes on the `0xAA 0xAA` sync header. The device must be paired in Android Bluetooth settings first.

Both paths produce identical `BrainWaveData` output through the same `dataFlow`.

## BLE default

`NeuroSkySdk.connect()` uses BLE by default. To use BT Classic instead, pass `TransportMode.BT_CLASSIC` to `connect()`. Both transports produce the same `dataFlow` output.

# 3. Requirements

## Device requirements

| Component  | Minimum                                                        |
|------------|----------------------------------------------------------------|
| Android OS | Android 6.0 (API level 23)                                     |
| Kotlin     | 1.9+                                                           |
| Coroutines | <code>kotlinx-coroutines-android</code> 1.7+                   |
| Bluetooth  | BLE adapter (for BLE mode) or Classic BT (for BT Classic mode) |
| Pairing    | Not required for BLE; required for BT Classic                  |

## Supported headset

This SDK is designed and tested for the **NeuroSky MindWave Mobile 2**. Both BLE and BT Classic modes are supported.

# 4. Installation

## Step 1 — Add JitPack to your repository list

In `settings.gradle.kts`:

```
dependencyResolutionManagement {  
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)  
    repositories {  
        google()  
        mavenCentral()  
    }  
}
```

```
        maven { url = uri("https://jitpack.io") } // add this
    }
}
```

## Step 2 — Add the dependency

In `app/build.gradle.kts` :

```
dependencies {
    implementation("com.github.nsk-bci:mindwave-sdk-android:v2.0.1")
}
```

## Step 3 — Sync and verify

Android Studio → File → Sync Project with Gradle Files

Or via terminal:

```
./gradlew assembleDebug
```

If the build succeeds without errors, the SDK is correctly installed.

**Note:** JitPack builds the library on first request. The very first `./gradlew` run may take 30–60 seconds while JitPack fetches and compiles the source. Subsequent builds use the cached artifact.

## 5. Permissions

Android requires Bluetooth permissions in `AndroidManifest.xml` . The required permissions differ by Android version.

### AndroidManifest.xml

```
<!-- Android 12+ (API 31+) -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
    android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

<!-- Android 6.0-11 (API 23-30) -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Adding all of the above covers all Android versions from 6.0 to 14+. The OS ignores permissions not applicable to the current version.

### Runtime permission request (Android 12+)

On Android 12 and above, `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` are runtime permissions — the user must grant them at runtime, not just at install time.

```
// In your Activity or Fragment
private val bluetoothPermissionLauncher = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    val allGranted = permissions.values.all { it }
    if (allGranted) {
        startEegSession()
    } else {
        showPermissionDeniedMessage()
    }
}

private fun requestBluetoothPermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        bluetoothPermissionLauncher.launch(
            arrayOf(
                Manifest.permission.BLUETOOTH_SCAN,
                Manifest.permission.BLUETOOTH_CONNECT
            )
        )
    } else {
        // Android 11 and below: request location permission
        bluetoothPermissionLauncher.launch(
            arrayOf(Manifest.permission.ACCESS_FINE_LOCATION)
        )
    }
}
```

## Why location permission is needed (Android 6–11)

On Android versions before 12, BLE scanning is considered a location-revealing operation because BLE beacons can be used for indoor positioning. The OS therefore requires `ACCESS_FINE_LOCATION` to perform BLE scans — even if your app has nothing to do with location. This requirement was lifted in Android 12 with the introduction of `BLUETOOTH_SCAN`.

## 6. Quick Start

The following example is a complete minimal implementation inside an Activity using `lifecycleScope`:

```
import com.neurosky.sdk.NeuroSkySdk
import com.neurosky.sdk.NeuroSkyCommand
import com.neurosky.sdk.model.SignalQuality
import com.neurosky.sdk.transport.ConnectionState

class MainActivity : AppCompatActivity() {

    private lateinit var sdk: NeuroSkySdk

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```

// Step 1: Create SDK instance (requires Context)
sdk = NeuroSkySdk(this)

// Step 2: Observe connection state
lifecycleScope.launch {
    sdk.connectionState.collect { state ->
        when (state) {
            ConnectionState.SCANNING    -> updateStatus("Scanning...")
            ConnectionState.CONNECTING  -> updateStatus("Connecting..")
            ConnectionState.CONNECTED   -> updateStatus("Connected")
            ConnectionState.ERROR       -> updateStatus("Connection failed")
            ConnectionState.DISCONNECTED -> updateStatus("Disconnected")
        }
    }
}

// Step 3: Connect and stream data
lifecycleScope.launch {
    // Connect using device name (BLE by default)
    sdk.connect("MindWave Mobile")

    // Set notch filter for your region
    sdk.sendCommand(NeuroSkyCommand.NOTCH_60HZ) // Korea/USA
    // sdk.sendCommand(NeuroSkyCommand.NOTCH_50HZ) // Europe/China

    // Collect EEG data stream
    sdk.dataFlow.collect { data ->
        if (data.signalQuality == SignalQuality.NO_SIGNAL) {
            updateStatus("No signal - adjust headset")
            return@collect
        }

        attentionBar.progress = data.attention
        meditationBar.progress = data.meditation
        signalText.text       = data.signalQuality.name
    }
}

override fun onDestroy() {
    super.onDestroy()
    lifecycleScope.launch { sdk.disconnect() }
}
}

```

## Connecting by MAC address vs device name

`connect()` accepts either a device name or a Bluetooth MAC address:

```
// By device name (BLE scan required – slower, ~5 sec)
sdk.connect("MindWave Mobile")

// By MAC address (direct connect – faster, recommended for production)
sdk.connect("AA:BB:CC:DD:EE:FF")
```

Using the MAC address is faster and more reliable in production. Use device name during prototyping when you don't yet know the MAC.

## 7. EEG Data Model

`dataFlow` emits a `BrainWaveData` object for each packet received from the MindWave Mobile headset.

```
data class BrainWaveData(
    val timestamp: Long,           // System.currentTimeMillis() at receipt
    val poorSignal: Int,          // 0 = perfect, 200 = no contact
    val attention: Int,           // 0~100, eSense™ attention
    val meditation: Int,          // 0~100, eSense™ meditation
    val delta: Int,               // 0.5~2.75 Hz band power
    val theta: Int,              // 3.5~6.75 Hz band power
    val lowAlpha: Int,           // 7.5~9.25 Hz band power
    val highAlpha: Int,          // 10~11.75 Hz band power
    val lowBeta: Int,            // 13~16.75 Hz band power
    val highBeta: Int,           // 18~29.75 Hz band power
    val lowGamma: Int,           // 31~39.75 Hz band power
    val midGamma: Int,           // 41~49.75 Hz band power
    val rawEeg: List<Int>,        // 10 samples/packet, signed 16-bit, 512Hz
    val eyeBlink: Int,           // 0 = no blink, 1~255 = blink intensity
) {
    val signalQuality: SignalQuality // derived from poorSignal
}
```

### Data update rates

| Field(s)               | Update rate  | Notes                                  |
|------------------------|--------------|----------------------------------------|
| poorSignal             | ~1 Hz        | Every packet                           |
| attention, meditation  | ~1 Hz        | eSense™ computed once per second       |
| delta through midGamma | ~1 Hz        | FFT computed once per second           |
| rawEeg                 | 512 Hz       | 10 samples per packet, ~51 packets/sec |
| eyeBlink               | Event-driven | Only non-zero when blink detected      |

When `rawEeg` packets arrive, `attention`, `meditation`, and frequency band fields are `0` in that object — they only appear in the `eSense` packet which arrives separately. Filter by checking which fields are non-zero.

## 8. EEG Frequency Bands Explained

The ThinkGear chip performs a Fast Fourier Transform (FFT) on the raw EEG and outputs power in 8 frequency bands.

### Important: values are relative, not absolute

The frequency band values are **relative power** in arbitrary units — not calibrated to physical units ( $\mu\text{V}^2/\text{Hz}$ ). This means:

- You **cannot** compare values across different sessions or individuals in absolute terms
- You **can** compare values within a single session — e.g., "Delta rose 40% after eyes closed"
- **Ratios** between bands are more meaningful than raw values

### Band reference table

| Field     | Band           | Frequency   | Typical mental states                                                 |
|-----------|----------------|-------------|-----------------------------------------------------------------------|
| delta     | $\delta$ Delta | 0.5~2.75 Hz | Deep sleep, healing. High delta while awake = fatigue or poor signal. |
| theta     | $\theta$ Theta | 3.5~6.75 Hz | Drowsiness, daydreaming, creativity, REM sleep, deep meditation.      |
| lowAlpha  | $\alpha$ Low   | 7.5~9.25 Hz | Relaxed, unfocused, calm. Increases with eyes closed.                 |
| highAlpha | $\alpha$ High  | 10~11.75 Hz | Eyes-closed rest. Suppressed by active visual attention.              |
| lowBeta   | $\beta$ Low    | 13~16.75 Hz | Active focus, alert thinking. The "work" band.                        |
| highBeta  | $\beta$ High   | 18~29.75 Hz | High arousal, stress, anxiety, intense cognition.                     |
| lowGamma  | $\gamma$ Low   | 31~39.75 Hz | Higher cognition, cross-modal perception, binding.                    |
| midGamma  | $\gamma$ Mid   | 41~49.75 Hz | Intense concentration. Elevated in expert meditators.                 |

### eSense™ Attention and Meditation

These are NeuroSky's **proprietary processed values** computed by the ThinkGear chip itself. The SDK receives them pre-computed.

**Attention (0~100)** — mental focus or concentration:

| Range | Meaning                                   |
|-------|-------------------------------------------|
| 0     | Not yet computed (startup, or no signal)  |
| 1~40  | Low — distracted, relaxed, wandering mind |
| 40~60 | Neutral baseline                          |
| 60~80 | Moderate focus — engaged in task          |



|        |                                          |
|--------|------------------------------------------|
| 80~100 | High focus — strong active concentration |
|--------|------------------------------------------|

**Meditation (0~100)** — mental calmness or relaxation:

| Range  | Meaning                                  |
|--------|------------------------------------------|
| 0      | Not yet computed (startup, or no signal) |
| 1~40   | Low — active thinking, stress            |
| 40~60  | Neutral baseline                         |
| 60~80  | Moderate relaxation                      |
| 80~100 | Deep calm — strong meditative state      |

*eSense values require 10~20 seconds to stabilize after the headset is put on. Values of 0 at startup are normal.*

## Raw EEG

When Raw EEG is enabled, `rawEeg` contains 10 signed 16-bit samples per packet at 512 Hz:

```
// Enable raw EEG after connecting
sdk.sendCommand(NeuroSkyCommand.START_RAW_EEG)

sdk.dataFlow.collect { data ->
  data.rawEeg.forEach { sample ->
    // Each sample: -32768 to +32767
    // 512 Hz, 10 samples per packet
    plotSample(sample)
  }
}
```

Raw EEG is useful for custom FFT analysis, artifact detection, or research. It is **disabled by default** to reduce Bluetooth bandwidth.

## 9. Signal Quality

Signal quality is the most critical factor for usable data. Always check it before using attention, meditation, or band values.

### PoorSignal values

| Value  | signalQuality | Reliability | Action                          |
|--------|---------------|-------------|---------------------------------|
| 0      | GOOD          | Excellent   | Use all data freely             |
| 1~50   | FAIR          | Acceptable  | Minor noise, eSense still valid |
| 51~199 | POOR          | Unreliable  | Prompt user to adjust headset   |
| 200    | NO_SIGNAL     | No data     | Headset not worn                |

## Recommended check pattern

```
sdk.dataFlow.collect { data ->
    when (data.signalQuality) {
        SignalQuality.NO_SIGNAL -> {
            showMessage("Please put on the MindWave Mobile headset.")
            return@collect
        }
        SignalQuality.POOR -> {
            showMessage("Weak signal (${data.poorSignal}). Adjust the headset.")
            // Optionally still log, but don't drive UI decisions
            return@collect
        }
        SignalQuality.FAIR,
        SignalQuality.GOOD -> {
            // Data is usable – update UI
            updateAttentionUi(data.attention)
            updateMeditationUi(data.meditation)
        }
    }
}
```

## Tips for improving signal quality

1. **Moisten the sensor** — a small drop of water on the forehead sensor significantly improves conductance
2. **Clean the forehead** — remove sunscreen, makeup, or sweat residue
3. **Adjust headset position** — center the sensor on FP1 (above the left eyebrow)
4. **Check the ear clip** — must make firm contact with the earlobe
5. **Wait 20~30 seconds** — after putting on the headset for signal to stabilize
6. **Avoid strong muscle movement** — clenching the jaw creates EMG artifact that looks like a signal-quality drop

---

## 10. Commands

After connecting, send control commands to configure the MindWave Mobile headset's behavior.

### Notch filter

EEG signals are extremely low amplitude and easily contaminated by AC power-line noise. Send the notch filter command immediately after connecting:

| Region       | Grid frequency | Command                    |
|--------------|----------------|----------------------------|
| Korea        | 60 Hz          | NeuroSkyCommand.NOTCH_60HZ |
| USA / Canada | 60 Hz          | NeuroSkyCommand.NOTCH_60HZ |
| Europe       | 50 Hz          | NeuroSkyCommand.NOTCH_50HZ |
| China        | 50 Hz          | NeuroSkyCommand.NOTCH_50HZ |

|                |       |                            |
|----------------|-------|----------------------------|
| Australia / UK | 50 Hz | NeuroSkyCommand.NOTCH_50HZ |
|----------------|-------|----------------------------|

```

sdk.connect("MindWave Mobile")
sdk.sendCommand(NeuroSkyCommand.NOTCH_60HZ) // Korea/USA
// sdk.sendCommand(NeuroSkyCommand.NOTCH_50HZ) // Europe/China

```

Without the notch filter, you will likely see a large 50Hz or 60Hz artifact in raw EEG and elevated Beta band values.

## Raw EEG streaming

```

// Enable raw EEG (disabled by default)
sdk.sendCommand(NeuroSkyCommand.START_RAW_EEG)

// Disable when no longer needed
sdk.sendCommand(NeuroSkyCommand.STOP_RAW_EEG)

```

## eSense control

```

// Disable eSense if only raw EEG is needed
sdk.sendCommand(NeuroSkyCommand.STOP_ESENSE)

// Re-enable
sdk.sendCommand(NeuroSkyCommand.START_ESENSE)

```

## All commands

| Constant                      | Byte | Description            |
|-------------------------------|------|------------------------|
| NeuroSkyCommand.NOTCH_60HZ    | 0x1C | Notch filter at 60 Hz  |
| NeuroSkyCommand.NOTCH_50HZ    | 0x1B | Notch filter at 50 Hz  |
| NeuroSkyCommand.START_RAW_EEG | 0x15 | Enable raw EEG stream  |
| NeuroSkyCommand.STOP_RAW_EEG  | 0x16 | Disable raw EEG stream |
| NeuroSkyCommand.START_ESENSE  | 0x17 | Enable eSense output   |
| NeuroSkyCommand.STOP_ESENSE   | 0x18 | Disable eSense output  |

# 11. Simulator — Develop Without Hardware

`SimulatorTransport` generates synthetic EEG data without any MindWave Mobile hardware. It implements `Transport`, so your application code remains unchanged between development and production.

## Why use the Simulator

- **No hardware required** — build and test UI, data flows, and business logic immediately

- **Predictable data** — use `FOCUSED` mode to always generate high-attention values for UI testing
- **Edge case testing** — `POOR_SIGNAL` mode tests your error-handling and reconnect logic
- **CI/CD pipelines** — run Espresso or unit tests on build servers without Bluetooth hardware

## Basic usage

```
import com.neurosky.sdk.simulator.SimulatorTransport

val simulator = SimulatorTransport()
simulator.setMode(SimulatorTransport.Mode.FOCUSED)

lifecycleScope.launch {
    simulator.connect("simulator") // any string accepted

    simulator.dataFlow.collect { data ->
        Log.d("SIM", "Attention: ${data.attention}, Meditation: ${data.meditation}")
    }
}
```

## Simulator modes

| Mode        | Attention      | Meditation     | PoorSignal | Use case                       |
|-------------|----------------|----------------|------------|--------------------------------|
| RANDOM      | 0~100 (random) | 0~100 (random) | 0          | General integration testing    |
| FOCUSED     | 70~100         | 40~60          | 0          | High-attention UI testing      |
| RELAXED     | 20~50          | 70~100         | 0          | High-meditation UI testing     |
| POOR_SIGNAL | 0              | 0              | 150~200    | Signal loss and error handling |

## Switching modes at runtime

```
val simulator = SimulatorTransport()
simulator.setMode(SimulatorTransport.Mode.POOR_SIGNAL)

lifecycleScope.launch {
    simulator.connect("simulator")

    // Simulate signal recovery after 5 seconds
    delay(5000)
    simulator.setMode(SimulatorTransport.Mode.FOCUSED)

    simulator.dataFlow.collect { data -> /* ... */ }
}
```

## Using in ViewModel with dependency injection

```
// Swap between simulator and real SDK via DI or build flavor
class EgViewModel {
```

```

    private val transport: Transport // injected: NeuroSkySdk or SimulatorTransport
) : ViewModel() {

    val data: Flow<BrainWaveData> = transport.dataFlow
    val state: Flow<ConnectionState> = transport.stateFlow

    fun connect(address: String) {
        viewModelScope.launch {
            transport.connect(address)
        }
    }
}

```

## 12. Error Handling & Reconnection

### Connection errors

`connect()` is a suspend function — wrap in try/catch for production use:

```

lifecycleScope.launch {
    try {
        sdk.connect("MindWave Mobile")
    } catch (e: SecurityException) {
        // Bluetooth permission not granted
        showError("Bluetooth permission required.")
    } catch (e: Exception) {
        // BT adapter off, device not found, etc.
        showError("Connection failed: ${e.message}")
    }
}

```

### Auto-reconnect loop

`dataFlow` completes (collection ends) when the connection drops. Wrap in a retry loop for production:

```

lifecycleScope.launch {
    while (isActive) { // isActive = CoroutineScope is still alive
        try {
            sdk.connect("MindWave Mobile")
            sdk.sendCommand(NeuroSkyCommand.NOTCH_60HZ)

            sdk.dataFlow.collect { data ->
                processData(data)
            }

            // dataFlow completed – headset turned off
            updateStatus("Disconnected. Reconnecting...")

        } catch (e: CancellationException) {
            break // ViewModel/lifecycle scope cancelled – exit cleanly
        }
    }
}

```

```

        } catch (e: Exception) {
            updateStatus("Error: ${e.message}. Retrying in 3s...")
        }

        delay(3000) // wait before retry
    }
}

```

## Handling NO\_SIGNAL without disconnecting

The Bluetooth connection can remain active while the electrode is not touching the skin. In this case, `signalQuality` becomes `NO_SIGNAL` but `dataFlow` keeps emitting. Handle this in your collector:

```

sdk.dataFlow.collect { data ->
    if (data.signalQuality == SignalQuality.NO_SIGNAL) {
        updateUi(connected = true, signalOk = false)
        return@collect
    }
    updateUi(connected = true, signalOk = true)
    processData(data)
}

```

## 13. Advanced Patterns

### ViewModel + StateFlow for Compose or XML UI

```

class EggViewModel(application: Application) : AndroidViewModel(application) {

    private val sdk = NeuroSkySdk(application)

    private val _attention = MutableStateFlow(0)
    private val _meditation = MutableStateFlow(0)
    private val _signal = MutableStateFlow(SignalQuality.NO_SIGNAL)

    val attention: StateFlow<Int> = _attention.asStateFlow()
    val meditation: StateFlow<Int> = _meditation.asStateFlow()
    val signal: StateFlow<SignalQuality> = _signal.asStateFlow()
    val state: StateFlow<ConnectionState> = sdk.connectionState

    init {
        viewModelScope.launch {
            sdk.connect("MindWave Mobile")
            sdk.sendCommand(NeuroSkyCommand.NOTCH_60HZ)

            sdk.dataFlow.collect { data ->
                _signal.value = data.signalQuality
                _attention.value = data.attention
                _meditation.value = data.meditation
            }
        }
    }
}

```

```

    }
}

override fun onCleared() {
    viewModelScope.launch { sdk.disconnect() }
}
}

```

## Jetpack Compose integration

```

@Composable
fun EegScreen(viewModel: EegViewModel = viewModel()) {
    val attention by viewModel.attention.collectAsState()
    val meditation by viewModel.meditation.collectAsState()
    val signal by viewModel.signal.collectAsState()

    Column(modifier = Modifier.padding(16.dp)) {
        Text("Signal: $signal")
        LinearProgressIndicator(progress = attention / 100f)
        Text("Attention: $attention")
        LinearProgressIndicator(progress = meditation / 100f)
        Text("Meditation: $meditation")
    }
}

```

## Buffering 1 second of raw EEG for custom analysis

```

// Raw EEG arrives as 10 samples per packet.
// Accumulate 512 samples for 1 second of data at 512 Hz.

val buffer = mutableListOf<Int>()

sdk.sendCommand(NeuroSkyCommand.START_RAW_EEG)

sdk.dataFlow.collect { data ->
    if (data.rawEeg.isEmpty()) return@collect

    buffer.addAll(data.rawEeg)

    if (buffer.size >= 512) {
        val oneSecond = buffer.take(512).toIntArray()
        buffer.subList(0, 512).clear()

        // Run your own FFT or signal processing
        val spectrum = MyFft.compute(oneSecond, sampleRate = 512)
        displaySpectrum(spectrum)
    }
}

```

## Recording session data to a file

```
val filename = "eeg_${System.currentTimeMillis()}.csv"
val file = File(getExternalFilesDir(null), filename)

file.bufferedWriter().use { writer ->
    writer.write("timestamp,attention,meditation,poor_signal,delta,theta," +
        "low_alpha,high_alpha,low_beta,high_beta,low_gamma,mid_gamma\n")

    sdk.dataFlow
        .filter { it.attention > 0 || it.meditation > 0 } // skip raw-only packets
        .collect { d ->
            writer.write("${d.timestamp},${d.attention},${d.meditation},${d.poorSignal}," +
                "${d.delta},${d.theta},${d.lowAlpha},${d.highAlpha}," +
                "${d.lowBeta},${d.highBeta},${d.lowGamma},${d.midGamma}\n")
        }
}
```

## Background service with foreground notification

For applications that need to stream EEG data while the app is in the background:

```
class EegForegroundService : Service() {

    private val sdk by lazy { NeuroSkySdk(this) }
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        startForeground(NOTIFICATION_ID, buildNotification())

        scope.launch {
            sdk.connect("MindWave Mobile")
            sdk.dataFlow.collect { data ->
                broadcastData(data) // send to Activity via LocalBroadcastManager
            }
        }

        return START_STICKY
    }

    override fun onDestroy() {
        scope.cancel()
        super.onDestroy()
    }
}
```

---

## 14. Troubleshooting

### Connection issues



| Symptom                                                  | Likely cause                                | Solution                                                                                                                                  |
|----------------------------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SecurityException</code> on <code>connect()</code> | Bluetooth permission not granted at runtime | Request <code>BLUETOOTH_SCAN</code> + <code>BLUETOOTH_CONNECT</code> (API 31+) or <code>ACCESS_FINE_LOCATION</code> (API 23–30)           |
| <code>connect()</code> hangs indefinitely                | BLE scan timeout                            | Use <code>findDeviceAddress()</code> first to resolve the MAC address, then pass the MAC to <code>connect()</code> for faster connection. |
| BT Classic connect fails                                 | Device not paired                           | Open Android Settings → Bluetooth → pair "MindWave Mobile" manually                                                                       |
| Connection drops after a few minutes                     | Android BLE background scan kill            | Use a Foreground Service to prevent the OS from killing the connection                                                                    |
| <code>dataFlow</code> stops emitting after screen off    | Background execution limit                  | Same — use a Foreground Service                                                                                                           |

### Signal quality issues

| Symptom                                                     | Likely cause                          | Solution                                                                      |
|-------------------------------------------------------------|---------------------------------------|-------------------------------------------------------------------------------|
| <code>NO_SIGNAL</code> immediately after connecting         | Electrode not touching forehead       | Adjust headset and press sensor firmly to skin                                |
| <code>POOR</code> signal persists after 30 seconds          | Dry skin or dirty sensor              | Wet the sensor tip with water; clean forehead                                 |
| <code>attention</code> and <code>meditation</code> always 0 | eSense needs warm-up                  | Wait 20–30 seconds after achieving <code>GOOD</code> signal                   |
| 50Hz or 60Hz spike in raw EEG                               | No notch filter                       | Call <code>sendCommand(NeuroSkyCommand.NOTCH_60HZ)</code> after connecting    |
| <code>rawEeg</code> always empty                            | <code>START_RAW_EEG</code> not called | Call <code>sendCommand(NeuroSkyCommand.START_RAW_EEG)</code> after connecting |

### Build issues

| Symptom                                                                | Likely cause                   | Solution                                                                                       |
|------------------------------------------------------------------------|--------------------------------|------------------------------------------------------------------------------------------------|
| Could not resolve <code>com.github.nsk-bci:mindwave-sdk-android</code> | JitPack not in repository list | Add <code>maven { url = uri("https://jitpack.io") }</code> to <code>settings.gradle.kts</code> |
| Unresolved reference: <code>NeuroSkySdk</code>                         | Missing import                 | Add <code>import com.neurosky.sdk.NeuroSkySdk</code>                                           |
| First build very slow                                                  | JitPack building from source   | Normal — only happens once; subsequent builds use cache                                        |

## 15. Testing

The SDK ships with a unit test suite for `ThinkGearParser` — the packet parser that runs identically on both BLE and BT Classic transports. These tests require no hardware or Bluetooth adapter and run on the JVM directly.

### Running the tests

```
./gradlew :sdk:test
```

Test results are written to:

```
sdk/build/reports/tests/testDebugUnitTest/index.html
```

### What is covered

| Test                         | Description                                      |
|------------------------------|--------------------------------------------------|
| parseEsense_0xEA             | Attention, Meditation, PoorSignal extraction     |
| parseEsense_0xEA_tooShort    | Short packet → returns null                      |
| parseEsense_0xEB             | Delta, Theta, LowAlpha, HighAlpha extraction     |
| parseEsense_0xEC             | LowBeta, HighBeta, LowGamma, MidGamma extraction |
| parseRawEeg_returns10Samples | 20-byte raw EEG → 10 signed int samples          |
| parseRawEeg_signedConversion | Values > 32768 converted to negative             |
| parseRawEeg_tooShort         | Short packet → returns null                      |
| parse_unknownUuid            | Unknown UUID → returns null                      |
| parseByte_validPacket        | BT Classic serial packet — Attention/Meditation  |
| parseByte_invalidChecksum    | Wrong checksum → returns null                    |
| parseByte_poorSignalCode     | BT Classic PoorSignal (code 0x02)                |
| signalQuality_*              | 200/100/25/0 → NO_SIGNAL/POOR/FAIR/GOOD          |

### Test location

```
sdk/src/test/kotlin/com/neurosky/sdk/parser /
└── ThinkGearParserTest.kt
```

## 16. API Reference

### NeuroSkySdk

Main entry point. Manages BLE/BT Classic transport selection and lifecycle.

```
class NeuroSkySdk(context: Context)
```

| Member | Type | Description |
|--------|------|-------------|
|--------|------|-------------|

|                        |                            |                                                                           |
|------------------------|----------------------------|---------------------------------------------------------------------------|
| connectionState        | StateFlow<ConnectionState> | Current connection state; hot Flow, always has a value                    |
| dataFlow               | Flow<BrainWaveData>        | Cold Flow of EEG packets; collect to start receiving                      |
| connect(deviceAddress) | suspend fun                | Connects via BLE by default; pass TransportMode.BT_CLASSIC for BT Classic |
| disconnect()           | suspend fun                | Gracefully closes the active transport                                    |
| sendCommand(cmd: Byte) | suspend fun                | Sends a control byte to the headset                                       |

### BrainWaveData

Immutable data class emitted by `dataFlow`.

| Field         | Type          | Range        | Description                           |
|---------------|---------------|--------------|---------------------------------------|
| timestamp     | Long          | —            | System.currentTimeMillis() at receipt |
| poorSignal    | Int           | 0~200        | 0 = perfect contact, 200 = no contact |
| attention     | Int           | 0~100        | eSense™ attention (0 = not computed)  |
| meditation    | Int           | 0~100        | eSense™ meditation (0 = not computed) |
| delta         | Int           | 0~∞          | Delta band power, 0.5~2.75 Hz         |
| theta         | Int           | 0~∞          | Theta band power, 3.5~6.75 Hz         |
| lowAlpha      | Int           | 0~∞          | Low Alpha, 7.5~9.25 Hz                |
| highAlpha     | Int           | 0~∞          | High Alpha, 10~11.75 Hz               |
| lowBeta       | Int           | 0~∞          | Low Beta, 13~16.75 Hz                 |
| highBeta      | Int           | 0~∞          | High Beta, 18~29.75 Hz                |
| lowGamma      | Int           | 0~∞          | Low Gamma, 31~39.75 Hz                |
| midGamma      | Int           | 0~∞          | Mid Gamma, 41~49.75 Hz                |
| rawEeg        | List<Int>     | -32768~32767 | 512Hz ADC samples (10 per packet)     |
| eyeBlink      | Int           | 0~255        | Eye blink intensity; 0 = no blink     |
| signalQuality | SignalQuality | enum         | Derived from poorSignal               |

### ConnectionState

```
enum class ConnectionState { DISCONNECTED, SCANNING, CONNECTING, CONNECTED, ERROR }
```

| Value | Meaning |
|-------|---------|
|-------|---------|

|              |                                       |
|--------------|---------------------------------------|
| DISCONNECTED | No active connection                  |
| SCANNING     | Scanning for the target device        |
| CONNECTING   | Device found, establishing connection |
| CONNECTED    | Data stream active                    |
| ERROR        | Connection attempt failed             |

## SignalQuality

Derived from `BrainWaveData.poorSignal`.

| Value     | poorSignal | Reliability | Recommended action            |
|-----------|------------|-------------|-------------------------------|
| GOOD      | 0          | Excellent   | Use all data                  |
| FAIR      | 1~50       | Acceptable  | Use data; minor noise present |
| POOR      | 51~199     | Unreliable  | Prompt user to adjust headset |
| NO_SIGNAL | 200        | No data     | Prompt user to put on headset |

## Transport (interface)

Common interface implemented by `BleTransport`, `BtClassicTransport`, and `SimulatorTransport`.

```
interface Transport {
    val dataFlow: Flow<BrainWaveData>
    val stateFlow: Flow<ConnectionState>
    suspend fun connect(deviceAddress: String)
    suspend fun disconnect()
    suspend fun sendCommand(cmd: Byte)
}
```

## SimulatorTransport

```
class SimulatorTransport : Transport
```

| Member                           | Description                                                 |
|----------------------------------|-------------------------------------------------------------|
| <code>setMode(mode: Mode)</code> | Change simulation mode; takes effect on next emitted packet |
| <code>Mode.RANDOM</code>         | Random Attention and Meditation values each tick            |
| <code>Mode.FOCUSED</code>        | Attention 70~100, Meditation 4~60                           |
| <code>Mode.RELAXED</code>        | Attention 20~50, Meditation 7~100                           |
| <code>Mode.POOR_SIGNAL</code>    | PoorSignal 150~200, Attention 0, Meditation 0               |

---

## NeuroSkyCommand

`object` NeuroSkyCommand

| Constant      | Byte | When to use                                      |
|---------------|------|--------------------------------------------------|
| NOTCH_60HZ    | 0x1C | Power grid is 60 Hz (Korea, USA)                 |
| NOTCH_50HZ    | 0x1B | Power grid is 50 Hz (Europe, China)              |
| START_RAW_EEG | 0x15 | Enable raw EEG waveform (disabled by default)    |
| STOP_RAW_EEG  | 0x16 | Disable raw EEG waveform                         |
| START_ESENSE  | 0x17 | Enable Attention/Meditation (enabled by default) |
| STOP_ESENSE   | 0x18 | Disable Attention/Meditation                     |